

# Semana 4: Estructuras de datos fundamentales — listas, diccionarios, tuplas, conjuntos

**Nivel:** Básico | **Prerrequisitos:** Semana 3 (funciones, `def / return`, `for / range()`, `break / continue`), Semana 2 (variables, tipos de datos, operadores), Semana 1 (secuencia/decisión/repetición) | **Versión:** v1.0 | **Fecha:** 2026-09-14

## Objetivos de aprendizaje

1. Crear, indexar y modificar listas ( `list` ) usando índices positivos y negativos, slicing, y los métodos `append()`, `insert()`, `remove()`, `pop()` y `sort()`.
2. Crear y recorrer diccionarios ( `dict` ) como colecciones de pares clave-valor, usando `dict.get()` para evitar errores, y los métodos `keys()`, `values()` e `items()`.
3. Explicar por qué las tuplas ( `tuple` ) son inmutables, y usar el empaquetado/desempaquetado de tuplas (tuple unpacking), conectándolo con el `return` de múltiples valores visto en la Semana 3.
4. Crear conjuntos ( `set` ) para eliminar duplicados automáticamente, y aplicar las operaciones de unión ( `|` ), intersección ( `&` ) y diferencia ( `-` ).
5. Elegir la estructura de datos correcta para un problema dado, según si necesita orden, permitir duplicados, ser mutable, o acceder por clave en vez de por posición.

## 1. Listas: la estructura que ya usabas, ahora a fondo

Desde la Semana 1 usamos listas como `numeros = [4, 8, -3, 10, -7]` para recorrerlas con `for`. Una **lista** es, en esencia, una fila numerada de casilleros: cada valor tiene una posición (**índice**), empezando en `0`, y se puede agregar, quitar o reordenar sin crear una lista nueva desde cero.

```
frutas = ["manzana", "pera", "uva"]

print(frutas[0])    # manzana – primer elemento, índice 0
print(frutas[-1])   # uva – último elemento, índice -1 (contar desde el final)
print(frutas[1:3])  # ['pera', 'uva'] – slicing: desde el índice 1 hasta antes del 3
```

**Analogía: el tren con vagones numerados.** Cada vagón (elemento) tiene un número de vagón (índice) contado desde la locomotora (`0`, `1`, `2`, ...), pero también se puede contar desde el último vagón hacia adelante (`-1` es el último, `-2` el penúltimo). El slicing ( `frutas[1:3]` ) es "quiero ver del vagón 1 al 2, sin incluir el 3" — igual que `range(1, 3)` de la Semana 3 nunca incluye el número final.

### Modificar una lista: los métodos más usados

| MÉTODO                     | QUÉ HACE                                             | ANALOGÍA                                      |
|----------------------------|------------------------------------------------------|-----------------------------------------------|
| <code>.append(x)</code>    | agrega <code>x</code> al final                       | Subir un vagón nuevo al final del tren.       |
| <code>.insert(i, x)</code> | inserta <code>x</code> en la posición <code>i</code> | Meter un vagón nuevo entre dos ya existentes. |

| MÉTODO                  | QUÉ HACE                                                                                                 | ANALOGÍA                                            |
|-------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <code>.remove(x)</code> | quita la <b>primera</b> aparición del valor <code>x</code>                                               | Desenganchar el primer vagón con ese nombre.        |
| <code>.pop(i)</code>    | quita y <b>devuelve</b> el elemento en la posición <code>i</code> (sin <code>i</code> , quita el último) | Desenganchar un vagón y quedarte con él en la mano. |
| <code>.sort()</code>    | reordena la lista de menor a mayor (in-place)                                                            | Reordenar los vagones por tamaño.                   |

```

frutas = ["manzana", "pera", "uva"]
frutas.append("kiwi")          # ['manzana', 'pera', 'uva', 'kiwi']
frutas.insert(1, "naranja")    # ['manzana', 'naranja', 'pera', 'uva', 'kiwi']
frutas.remove("pera")          # ['manzana', 'naranja', 'uva', 'kiwi']
ultima = frutas.pop()          # ultima = 'kiwi'; frutas = ['manzana', 'naranja', 'uva']

numeros = [4, 8, -3, 10, -7]
numeros.sort()                 # [-7, -3, 4, 8, 10] – modifica la lista original
print(len(numeros))            # 5 – len() ya se usaba desde la Semana 3

```

**Punto de confusión clásico:** `.sort()` modifica la lista original y no devuelve nada útil (devuelve `None`); si necesitas conservar el orden original y obtener una lista nueva ya ordenada, se usa la función `sorted(numeros)`, que sí devuelve una lista nueva.

## List comprehensions: construir listas en una línea

En la Semana 3 recorrías una lista con `for` para transformarla o filtrarla, acumulando resultados en una lista vacía. Una **list comprehension** expresa exactamente esa misma idea en una sola línea:

```

numeros = [4, 8, -3, 10, -7]

# Con for tradicional (Semana 3):
positivos = []
for n in numeros:
    if n > 0:
        positivos.append(n)

# La misma idea como list comprehension:
positivos = [n for n in numeros if n > 0]
print(positivos)  # [4, 8, 10]

al_cuadrado = [n ** 2 for n in numeros]
print(al_cuadrado)  # [16, 64, 9, 100, 49]

```

No es una sintaxis distinta que aprender desde cero — es el mismo `for` + `if` de siempre, leído como "la lista de `n` para cada `n` en `numeros`, tal que `n > 0`".

---

## 2. Diccionarios: buscar por nombre, no por posición

Una lista funciona bien cuando el orden importa y se accede por posición. Pero para guardar los datos de una persona (nombre, edad, ciudad) no tiene sentido recordar "el dato en la posición 2 es la ciudad" — es mejor ponerle **nombre** a cada dato. Eso es un **diccionario**: una colección de pares `clave: valor`.

**Analogía: la agenda de contactos del teléfono.** No buscas a alguien por "la persona número 47 de mi lista" — buscas por su nombre (la clave) y el teléfono aparece (el valor). Un diccionario funciona igual: `agenda["Ana"]` devuelve el valor asociado a la clave `"Ana"`, sin importar en qué posición esté guardado internamente.

```
estudiante = {
    "nombre": "Ana",
    "edad": 21,
    "ciudad": "Bogotá",
}

print(estudiante["nombre"]) # Ana
estudiante["edad"] = 22     # actualiza un valor existente
estudiante["carrera"] = "Ingeniería" # agrega una clave nueva
print(estudiante)
```

`.get()` en vez de `[]`, para evitar errores: acceder con `estudiante["telefono"]` cuando esa clave no existe produce `KeyError` y detiene el programa. `.get()` devuelve `None` (o un valor por defecto que tú elijas) en vez de fallar:

```
print(estudiante.get("telefono")) # None – no existe, pero no falla
print(estudiante.get("telefono", "sin dato")) # "sin dato" – valor por defecto explícito
```

### Recorrer un diccionario:

```
for clave, valor in estudiante.items():
    print(f"{clave}: {valor}")

print(list(estudiante.keys())) # ['nombre', 'edad', 'ciudad', 'carrera']
print(list(estudiante.values())) # ['Ana', 22, 'Bogotá', 'Ingeniería']
```

`estudiante.items()` devuelve pares `(clave, valor)` que se pueden desempaquetar directamente en el `for` — esta idea de desempaquetar varios valores a la vez se explica a fondo en la siguiente sección con tuplas.

---

## 3. Tuplas: cuando los datos no deben cambiar

Una **tupla** se ve casi igual que una lista, pero se escribe con paréntesis `()` en vez de corchetes `[]`, y tiene una diferencia fundamental: es **inmutable** — una vez creada, no se puede agregar, quitar ni reemplazar ningún elemento.

```
coordenada = (4.7110, -74.0721) # latitud, longitud de un punto fijo
print(coordenada[0])           # 4.711

coordenada[0] = 0 # ERROR: TypeError – las tuplas no se pueden modificar
```

**Analogía: la fecha de nacimiento.** Una lista es como una lista de tareas pendientes: se espera que cambie, se agreguen o tachen elementos. Una tupla es como la fecha de nacimiento de una persona: es un dato que, por su propia naturaleza, no debería cambiar nunca después de creado. Python usa tuplas quien quiere decir "estos valores van juntos y así se quedan": coordenadas GPS, una fecha `(2026, 9, 14)`, o un par `(nombre, precio)` que representa un solo producto.

## Tuple unpacking (desempaquetado)

Ya usaste esto sin llamarlo por su nombre: en la Semana 3, `evaluar_promedio(notas)` hacía `return promedio, aprobado` — eso es literalmente devolver **una tupla** de dos valores, y `promedio, aprobado = evaluar_promedio(notas)` es **desempaquetarla** en dos variables de una sola vez.

```
def evaluar_promedio(notas):
    promedio = sum(notas) / len(notas)
    aprobado = promedio >= 6.0
    return promedio, aprobado # esto construye la tupla (promedio, aprobado)

resultado = evaluar_promedio([8.5, 7.0, 9.0])
print(resultado)             # (8.166666666666666, True) – es una tupla
print(type(resultado))       # <class 'tuple'>

promedio, aprobado = resultado # desempaquetado: una variable por posición
```

El mismo mecanismo funciona para intercambiar dos variables sin usar una variable temporal — algo que en otros lenguajes requiere tres líneas:

```
a, b = 1, 2
a, b = b, a # intercambio directo, sin variable auxiliar
print(a, b) # 2 1
```

## 4. Conjuntos: sin duplicados, sin orden garantizado

Un **conjunto** (`set`) guarda valores únicos: si intentas agregar un valor que ya existe, simplemente no pasa nada — no hay duplicados. Además, un conjunto no mantiene una posición fija para cada elemento (no se puede hacer `mi_set[0]`).

**Analogía: la lista de invitados de una fiesta.** Si alguien anota el mismo nombre dos veces por error, en la lista de invitados real esa persona sigue contando una sola vez. Un conjunto hace esa limpieza automáticamente.

```
colores_camisetas = ["rojo", "azul", "rojo", "verde", "azul", "rojo"]
colores_unicos = set(colores_camisetas)
print(colores_unicos) # {'rojo', 'azul', 'verde'} – sin duplicados, orden no garantizado
print(len(colores_unicos)) # 3
```

## Operaciones de conjuntos

```
salon_a = {"Ana", "Luis", "Marta"}
salon_b = {"Marta", "Pedro", "Ana"}

print(salon_a | salon_b)  # unión: {'Ana', 'Luis', 'Marta', 'Pedro'} – todos, sin repetir
print(salon_a & salon_b)  # intersección: {'Ana', 'Marta'} – están en ambos salones
print(salon_a - salon_b)  # diferencia: {'Luis'} – solo en salon_a, no en salon_b
```

Estas tres operaciones (`|`, `&`, `-`) son la traducción directa a código de "quién está en cualquiera de los dos grupos", "quién está en ambos" y "quién está en uno pero no en el otro" — preguntas que aparecen todo el tiempo al comparar datos.

## 5. Elegir la estructura correcta

| ESTRUCTURA         | ¿ORDENADA?                                | ¿PERMITE DUPLICADOS?  | ¿MUTABLE?                               | ¿ACCESO POR...?                            |
|--------------------|-------------------------------------------|-----------------------|-----------------------------------------|--------------------------------------------|
| <code>list</code>  | Sí                                        | Sí                    | Sí                                      | posición (índice)                          |
| <code>dict</code>  | Sí (orden de inserción, desde Python 3.7) | Claves no, valores sí | Sí                                      | clave                                      |
| <code>tuple</code> | Sí                                        | Sí                    | No                                      | posición (índice)                          |
| <code>set</code>   | No garantizado                            | No                    | Sí (se pueden agregar/quitar elementos) | pertenencia ( <code>x in conjunto</code> ) |

La pregunta que resuelve la mayoría de los casos: "¿necesito buscar por nombre/clave?" → `dict`. "¿el orden y las repeticiones importan, y puede cambiar?" → `list`. "¿son datos que van juntos y no deben cambiar?" → `tuple`. "¿solo me importa si algo está presente, sin repetidos?" → `set`.

## 6. Por qué esto importa para IA/ML

Estas cuatro estructuras son el lenguaje con el que se describen los datos en cualquier proyecto de IA/ML, antes incluso de tocar una sola biblioteca especializada:

- Un **DataFrame de pandas** (que verás en la Semana 9) es, conceptualmente, un diccionario donde cada clave es el nombre de una columna y cada valor es una lista con los datos de esa columna — exactamente la estructura `{"nombre": [...], "edad": [...]}`.
- Un **lote de entrenamiento** (batch) para una red neuronal casi siempre es una lista de ejemplos, y cada ejemplo suele ser una tupla `(entrada, etiqueta)` — los datos van juntos y no cambian mientras se procesan.
- El **vocabulario** de un modelo de lenguaje (la lista de palabras o tokens distintos que conoce) se construye típicamente pasando todo el texto por un `set`, para quedarse solo con las palabras únicas antes de numerarlas.
- Las **coordenadas de un embedding** (un vector que representa el significado de una palabra o imagen como números) se manejan como tuplas o arreglos de posiciones fijas, nunca como algo que se reordena a mitad de un cálculo.

## 7. Ejemplo práctico: analizador de un carrito de compras

**Problema:** dado un carrito de compras (una lista de productos, donde cada producto es un par nombre-precio), calcular el total a pagar, encontrar el producto más caro, y obtener la lista de categorías distintas presentes en la compra.

**Pseudocódigo:**

```
carrito = lista de tuplas (nombre, precio, categoria)

total = 0
PARA CADA producto EN carrito HACER
    total = total + precio del producto

producto_mas_caro = el producto con el precio máximo del carrito

categorias = conjunto vacío
PARA CADA producto EN carrito HACER
    agregar categoria del producto al conjunto de categorias

MOSTRAR total, producto_mas_caro, categorias
```

### Del pseudocódigo al código real (Python)

```
carrito = [
    ("laptop", 3200000, "tecnología"),
    ("mouse", 45000, "tecnología"),
    ("cuaderno", 8000, "papelería"),
    ("audífonos", 120000, "tecnología"),
]

# REPETICIÓN + acumulador: sumamos el precio (posición 1 de cada tupla)
total = 0
for nombre, precio, categoria in carrito: # tuple unpacking dentro del for
    total = total + precio

# Buscar el máximo con una key personalizada: max() recibe una función
# que le dice "compara los productos por su precio, no por el nombre".
producto_mas_caro = max(carrito, key=lambda producto: producto[1])

# set comprehension: la misma idea que una list comprehension, pero
# construyendo un conjunto – las categorías repetidas se eliminan solas.
categorias = {categoria for _, _, categoria in carrito}

print(f"Total a pagar: ${total:,}")
print(f"Producto más caro: {producto_mas_caro[0]} (${producto_mas_caro[1]:,})")
print(f"Categorías en la compra: {categorias}")
```

Resultado: Total a pagar: \$3,373,000 , Producto más caro: laptop (\$3,200,000) , Categorías en la compra: {'tecnología', 'papelería'} .

**Punto de confusión clásico:** `_` como nombre de variable en `for _, _, categoria in carrito` no es magia especial de Python — es solo una variable normal, usada por convención cuando su valor no se va a usar. Aquí dice explícitamente "no me importa el nombre ni el precio en esta línea, solo la categoría".

## Segundo ejemplo: agenda de contactos con diccionario de diccionarios

```
agenda = {
    "Ana": {"telefono": "3001112233", "ciudad": "Bogotá"},
    "Luis": {"telefono": "3004445566", "ciudad": "Medellín"},
}

# Agregar un contacto nuevo:
agenda["Marta"] = {"telefono": "3007778899", "ciudad": "Cali"}

# Buscar un dato sin arriesgarse a un KeyError si el contacto no existe:
contacto = agenda.get("Pedro")
if contacto is None:
    print("Pedro no está en la agenda")
else:
    print(contacto["telefono"])

# Recorrer toda la agenda:
for nombre, datos in agenda.items():
    print(f"{nombre} vive en {datos['ciudad']}")
```

## 8. Notas de vigencia técnica (2026)

`list`, `dict`, `tuple` y `set`, junto con sus métodos (`.append()`, `.get()`, `.items()`, etc.) son parte del núcleo del lenguaje desde siempre y no tienen reemplazo en camino. Dos detalles sí vale la pena adoptar desde ahora, porque son la forma moderna y no la de hace diez años: primero, desde Python 3.7 los diccionarios **garantizan** mantener el orden en que se insertaron las claves — antes era un detalle de implementación, no una garantía del lenguaje, así que código antiguo que "ordena manualmente" un diccionario para compensar eso ya es innecesario. Segundo, para anotar tipos (type hints, un tema que se profundiza más adelante) ya no se debe escribir `List[int]` importado de `typing` — desde Python 3.9 se escribe directamente `list[int]`, `dict[str, int]`, etc., sin importar nada adicional.

## 9. Errores comunes de principiante

- Confundir el **índice** (la posición, siempre un número) con el **valor** guardado en esa posición.
- Intentar modificar una tupla (`coordenada[0] = 0`) y no entender por qué falla — es su comportamiento esperado, no un error del programa.
- Usar `diccionario["clave"]` en vez de `diccionario.get("clave")` cuando no se está seguro de que la clave exista, y terminar con un `KeyError` que detiene el programa.
- Esperar que un conjunto mantenga el orden en que se agregaron los elementos — un `set` no lo garantiza; si el orden importa, se necesita una lista.
- Olvidar que `.sort()` modifica la lista original y no devuelve nada usable, en vez de usar `sorted()` cuando se quiere conservar la lista original intacta.

---

## 10. Ejercicio propuesto

Escribe una función en Python llamada `resumen_inventario(productos)` que reciba una lista de tuplas `(nombre, cantidad, precio_unitario)` y devuelva un diccionario con tres claves:

```
"valor_total": la suma de (cantidad * precio_unitario) de todos los productos.
"producto_mayor_stock": el nombre del producto con más unidades en cantidad.
"categorias_bajo_stock": una lista con los nombres de los productos que tengan cantidad menor a 5.
```

Luego, prueba tu función con al menos cuatro productos distintos e imprime el diccionario resultante.

Pista: vas a necesitar un acumulador (Semana 3), `max()` con `key` (como en el ejemplo del carrito), una list comprehension con `if`, y construir el diccionario de salida con las tres claves al final — no imprimas nada dentro de la función, solo devuelve el diccionario.

---

## 11. Autoevaluación

- ¿Qué diferencia hay entre `frutas[1]` y `frutas[1:3]`? — `frutas[1]` devuelve un único elemento (el de la posición 1); `frutas[1:3]` devuelve una lista nueva con los elementos desde la posición 1 hasta antes de la 3 (slicing).
  - ¿Qué hace `.pop()` que no hace `.remove()`? — `.pop()` quita un elemento por posición y **devuelve** ese valor; `.remove()` quita un elemento por su valor y no devuelve nada útil.
  - ¿Por qué `diccionario.get("clave")` es más seguro que `diccionario["clave"]`? — Porque si la clave no existe, `.get()` devuelve `None` (o un valor por defecto) en vez de detener el programa con `KeyError`.
  - ¿Qué significa que una tupla sea inmutable? — Que una vez creada, no se puede agregar, quitar ni reemplazar ninguno de sus elementos.
  - Dado `promedio, aprobado = evaluar_promedio(notas)`, ¿qué tipo de dato devuelve realmente `evaluar_promedio`? — Una tupla de dos valores, que luego se desempaqueta en las dos variables.
  - ¿Qué imprime `{1, 2, 2, 3, 3, 3}`? — `{1, 2, 3}` — un conjunto elimina automáticamente los valores duplicados.
  - ¿Qué operación de conjuntos usarías para saber qué estudiantes están inscritos en dos cursos a la vez? — La intersección (`&`).
  - V/F: "Los diccionarios en Python garantizan mantener el orden de inserción de sus claves desde Python 3.7." — Verdadero.
  - Explica con tus propias palabras cómo se relaciona un diccionario de Python con una tabla de datos de un DataFrame de pandas, sin usar jerga de programación. — Un DataFrame se puede pensar como un diccionario donde cada clave es el nombre de una columna y su valor es la lista de datos de esa columna, igual que `{"nombre": [...], "edad": [...]}`, solo que con herramientas adicionales para analizarlo.
- 

## 12. Verificación de dependencias

Este contenido depende directamente de la Semana 3 (funciones con `return`, incluyendo el `return` de múltiples valores que aquí se reconoce como una tupla; `for` / `range()`; `break` / `continue` usados dentro de los ejemplos) y, a través de ella, de las Semanas 1 y 2 (variables, tipos de datos, operadores de comparación, los tres bloques de secuencia/decisión/repetición). No se introduce ningún concepto matemático o de programación no visto antes: las listas ya se usaban desde la Semana 1 como algo que se recorre, y aquí se profundiza en cómo crearlas y modificarlas; los diccionarios, tuplas y conjuntos son estructuras nuevas, pero se explican reutilizando el

vocabulario ya conocido de variables, `for` e `if`. Es prerequisite directo de la Semana 5 (algoritmos y complejidad básica: búsqueda, ordenamiento, Big O), que asume que ya se sabe crear, recorrer y modificar listas y diccionarios como las estructuras de datos que esos algoritmos van a procesar.